Operating Systems

ASSIGNMENT 1

CO1: *Apply the concepts of operating system types, structures and system calls to demonstrate their functions in various computing environments – history of operating systems, mainframe, desktop, multiprocessor, distributed, clustered, real-time and handheld systems, OS structure, services, system calls, system design, CPU states, memory hierarchy and I/O systems.*

Assignment Title: Evaluate and Justify the Operating-System Design of a Smart Hospital (Total Marks - 50)

Nature of Assignment: Individual
Duration: 1 Week
Submission Date: 04-11-2026
Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: https://github.com/BommadiArchana/CSA0407---Operating-System/tree/main/Assignments/CO1%20Assignment

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Operating Systems for a Smart Hospital

A 500-bed hospital is replacing its computer systems. It needs: bedside patient monitors that must raise an alarm within 10 milliseconds of an abnormal reading; nurses' handheld tablets; doctors' desktop workstations; a central database server for patient records that must never stop; an imaging server that processes MRI scans using many processors; and a link to two partner hospitals that share records.

The IT team has proposed using one general-purpose operating system everywhere. The clinical staff are concerned that alarms may be delayed, the database may stop during updates, and the tablets' batteries will not last a shift.

The number of devices may grow:

1 ward (40 monitors, 10 tablets) → 1 hospital (500 monitors, 120 tablets, 80 workstations) → a group of 3 hospitals sharing records

You are required to evaluate the requirements of every part of the system, judge which type and structure of operating system suits each, examine how programs obtain OS services through system calls, and defend a complete operating-system plan for the hospital.

Design Challenge:

The hospital is adding remote patient monitoring at home and a cloud-based records service.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate the Requirements of Each Computing Environment — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate what each part of the hospital needs from its operating system.

Discuss:

- Response-time, reliability, throughput, battery and security requirements of each device
- Which requirements conflict (for example: fast response and many users on one machine)
- Which requirements are critical to patient safety
- How the lessons from the history of operating systems (batch, time-sharing, personal, mobile) apply to each environment

You are expected to **rank the requirements of each environment and justify the ranking.**

Alternative Concept and Complexity Analysis

If the whole hospital used a single general-purpose operating system on every device, instead of a different type of OS for each environment, what would the solution be? Evaluate the worst-case alarm response time (unbounded scheduling delay versus a guaranteed deadline), the memory needed on each device, and the administration effort.

Evidence Expected

A ranked requirement table for every environment with a justification for each ranking.

TASK 2 – Judge the Type of Operating System for Each Environment — 3 Marks

Bloom's Level: K5 – Evaluate

Compare the types of operating systems for the hospital.

Discuss:

- Mainframe, desktop, multiprocessor, distributed, clustered, real-time (hard and soft) and handheld systems
- Which type best fits the monitors, tablets, workstations, database server, imaging server and partner link
- Symmetric versus asymmetric multiprocessing for the imaging server
- A clustered system with fail-over for the database server

Alternative Concept and Complexity Analysis

If the database server used a single large multiprocessor machine instead of a cluster of smaller machines with fail-over, what would the solution be? Evaluate the availability (a single point of failure versus fail-over within seconds), the cost, and how throughput grows with the number of processors (time) and the memory available (space).

Evidence Expected

A justified choice of operating-system type for each environment, with the rejected alternatives and reasons.

TASK 3 – Evaluate Operating-System Structures — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate the possible structures of the operating system.

Discuss:

- Simple (monolithic), layered, microkernel, modular (loadable kernel modules) and hybrid structures
- The effect of each structure on reliability, performance and ease of updating
- Which structure suits the patient monitors and which suits the database server
- How Linux, Windows, Android and a real-time OS such as FreeRTOS or QNX are structured

Alternative Concept and Complexity Analysis

If the bedside monitors used a monolithic kernel instead of a microkernel, what would the solution be? Evaluate the time cost of a service request (one system call versus message passing between servers), the memory taken by the kernel, and the effect of a faulty driver on the whole system.

Evidence Expected

A comparison of structures against the hospital's requirements, with a defended choice for two environments.

TASK 4 – Evaluate the Services an Operating System Must Provide — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate the services each environment needs.

Discuss:

- User interface, program execution, I/O operations, file systems, communication, error detection, resource allocation, accounting, protection and security
- Which services can be removed from the bedside monitor to make it smaller and faster
- Which services are essential for the database server
- How the OS protects one patient's records from another user's programs

Alternative Concept and Complexity Analysis

If the tablets used a full desktop operating system with every service enabled instead of a reduced handheld OS, what would the solution be? Evaluate the memory and storage used by the extra services (space), the start-up time and the battery life (time).

Evidence Expected

A service table for each environment with justified inclusions and exclusions.

TASK 5 – Examine System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Examine how a program obtains OS services.

Discuss:

- The categories of system calls (process control, file management, device management, information maintenance, communication, protection)
- What happens step by step when a program calls read(): library call, trap, mode switch, kernel work, return
- How parameters are passed to the kernel (registers, a block in memory, the stack)
- How strace shows the system calls made by a program

Alternative Concept and Complexity Analysis

If programs called the C library functions fopen, fread and fwrite instead of the open, read and write system calls directly, what would the solution be? Evaluate the number of system calls for a file of n bytes (about n / 4096 with library buffering versus n / b for a buffer of b bytes) and the extra buffer space used by the library.

Evidence Expected

A step-by-step explanation of a system call with strace evidence from a real program.

TASK 6 – Evaluate the Cost of System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the number of system calls affects performance.

Discuss:

- Why each system call costs much more than an ordinary function call
- How the buffer size changes the number of read() and write() calls when the database copies a file

- How library buffering (fread, fwrite) reduces the number of system calls
- The best buffer size for the hospital's backup program, using your measurements

Complete the Code

Complete TODO 1 below using the read() and write() system calls, and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

/* Copy src to dst with the read() and write() system calls, using a buffer of
   bufsize bytes. Returns the number of read() calls made, or -1 on error. */
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }
    long calls = 0;
    ssize_t n;
    // TODO 1: Loop: read up to bufsize bytes into buf (count every read call in calls);
    //   stop when read returns 0 (end of file) or -1; write exactly the n bytes that
    //   were read

    close(in);
    close(out);
    return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w");     /* a 10,000-byte test file */
    for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
    fclose(f);

    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i],
               copy_file("records.dat", "copy.dat", sizes[i]));
    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

Expected output:

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Run the program under strace -c and compare the number of read() calls reported with the program's output.

Alternative Concept and Complexity Analysis

If the backup program used memory-mapped files (mmap) instead of read() and write() with a buffer, what would the solution be? Evaluate the number of system calls (a few versus about n / b), the copying between kernel and user space, and the memory used for very large files.

Evidence Expected

Measured system-call counts and times for different buffer sizes, with a justified recommendation.

TASK 7 – Evaluate Process Control System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how a command shell on the database server starts and manages programs.

Discuss:

- What fork(), exec(), wait() and exit() each do
- How a shell runs a command, and how it obtains the command's exit status
- What a zombie and an orphan process are, and how the OS deals with them
- Why fork() followed by exec() is used instead of a single "create process" call

Complete the Code

Complete TODO 1 below using fork(), execlp() and waitpid(), and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("shell: running job 1\n");
    fflush(stdout);                         /* empty the buffer before fork() */
    // TODO 1: Create a child with fork(). In the child, replace the program with
    //   execlp("expr", "expr", "6", "*", "7", (char *)NULL) and exit(127) if exec
    //   fails. In the parent, wait for the child with waitpid() and print "job 1 exit
    //   status N" using WEXITSTATUS


    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid == 0) _exit(3);                 /* a child that fails with code 3 */
    waitpid(pid, &status, 0);
    printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

Expected output:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Explain why stdout is flushed before each fork(). Then remove the second fflush(stdout), change _exit(3) to exit(3), redirect the output to a file and explain why "shell: running job 2" now appears twice.

Alternative Concept and Complexity Analysis

If commands were run with posix_spawn() or with threads instead of fork() followed by exec(), what would the solution be? Evaluate the time to start 1,000 commands, the memory copied or shared (copy-on-write), and the isolation between commands.

Evidence Expected

A working program that runs commands like a shell, with process diagrams and a reasoned evaluation.

TASK 8 – Evaluate CPU States and Protection — 3 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the hardware protects the hospital system.

Discuss:

- User mode and kernel mode, privileged instructions and the mode bit
- Interrupts, traps and the timer interrupt
- What would happen to the monitors if any program could disable interrupts
- How the hardware stops one program from reading another program's memory

Alternative Concept and Complexity Analysis

If the monitors used polling (checking the sensor in a loop) instead of interrupts, what would the solution be? Evaluate the worst-case delay before an abnormal reading is noticed (the polling interval versus the interrupt latency) and the CPU time and power wasted by polling.

Evidence Expected

A reasoned explanation of CPU modes, interrupts and protection, applied to the monitors.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how data moves between storage levels and devices.

Discuss:

- The levels of the memory hierarchy (registers, cache, main memory, SSD, hard disk, network storage) with typical access times and sizes
- Where MRI images, patient records and monitor readings should be kept
- Programmed I/O, interrupt-driven I/O and DMA, and which suits the MRI scanner
- The effect of caching on the speed of record look-up

Alternative Concept and Complexity Analysis

If the MRI scanner transferred images with interrupt-driven I/O (one interrupt per word) instead of DMA, what would the solution be? Evaluate the number of interrupts and the CPU time for a 500 MB image, and the memory buffers needed in each method.

Evidence Expected

A justified data-placement plan and I/O method for each device, using typical access times.

TASK 10 – Measure and Compare — 4 Marks

Bloom's Level: K5 – Evaluate

Measure the behaviour of your programs on a Linux system.

Discuss:

- The number of system calls made by your copy program for different buffer sizes (strace -c)
- The time spent in user mode and in kernel mode (the time command)
- The time to create 1,000 processes with fork() and to create 1,000 threads
- How the measurements support or contradict your earlier judgements

Alternative Concept and Complexity Analysis

If threads were created instead of processes for the measurements, what would the result be? Evaluate the creation time for 1,000 threads versus 1,000 processes and the memory used by each, and explain the difference using what each one shares.

Evidence Expected

Measured results in tables, compared with expectations, with explained differences.

TASK 11 – Recommend and Defend the Operating-System Plan — 4 Marks

Bloom's Level: K5 – Evaluate

Recommend the complete operating-system plan for the hospital.

Discuss:

- The type and structure of operating system for each environment
- The services, protection and I/O methods each will use
- The limitations and risks of your plan
- Whether the plan remains suitable for the group of three hospitals

Your recommendation must be based on **evidence from your own analysis and measurements.**

Alternative Concept and Complexity Analysis

If the whole system were moved to virtual machines on two large servers, instead of separate physical machines for each environment, what would the solution be? Evaluate the effect on response time for the monitors, the memory and processor overhead of virtualisation, and the effect on reliability.

Evidence Expected

A technically justified plan supported by analysis and measurements.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design — 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital adds **remote monitoring devices in patients' homes** that send readings over the internet, and moves patient records to a **cloud service** shared with the partner hospitals.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Identify which operating-system types, services and protection mechanisms are affected.

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your plan can support home devices and cloud records, considering response time, reliability and security.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe the revised plan (for example: an embedded real-time OS on home devices, a distributed system for shared records).

Evidence Expected

An appropriate design response.

D. Evidence-Based Defence – 2 Marks

Defend your decision using evidence and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating evidence and the changed requirements.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Requirements of Each Environment	K5 – Evaluate	4
Task 2 – Types of Operating Systems	K5 – Evaluate	3
Task 3 – Operating-System Structures	K5 – Evaluate	4
Task 4 – Operating-System Services	K5 – Evaluate	4
Task 5 – System Calls	K5 – Evaluate	4
Task 6 – Cost of System Calls	K5 – Evaluate	4
Task 7 – Process Control System Calls	K5 – Evaluate	4
Task 8 – CPU States and Protection	K5 – Evaluate	3

Task	Bloom's Level	Marks
Task 9 – Memory Hierarchy and I/O	K5 – Evaluate	4
Task 10 – Measurement and Comparison	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

Submission Requirements

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including requirement tables, diagrams, strace and time outputs, measurements and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

Name:B.Archana

Reg No:192425325

ASSIGNMENT -1 (CO1)

TASK 1 – Evaluate the Requirements of Each Computing Environment

Ranked Requirements Table

Rank 1 indicates the highest priority, and Rank 5 indicates the lowest priority for that environment.

Computing environment	1st	2nd	3rd	4th	5th
Bedside patient monitors	Response time	Reliability	Security	Throughput	Battery life
Nurses' handheld tablets	Battery life	Security	Reliability	Response time	Throughput
Doctors' desktop workstations	Security	Response time	Reliability	Throughput	Battery life
Central database server	Reliability	Security	Throughput	Response time	Battery life
MRI imaging server	Throughput	Reliability	Response time	Security	Battery life
Distributed hospital network	Reliability	Security	Throughput	Response time	Battery life

Justification for Each Environment

1. Bedside patient monitors: An RTOS is suitable because abnormal readings must trigger alarms within 10 milliseconds. Predictable response time and reliability are critical for patient safety.

2. Nurses' handheld tablets: A mobile OS such as Android provides power management, wireless communication and secure access to patient records. Battery life is important for completing a full nursing shift.

3. Doctors' desktop workstations: A desktop OS such as Windows or Linux supports electronic health records, multitasking and medical applications. Security is essential because doctors access confidential patient information.

4. Central database server: A server OS such as Linux Server supports multiple users, efficient storage and secure record management. Redundancy and failover are necessary to minimize downtime.

5. MRI imaging server: A multiprocessor-capable OS supports parallel processing across CPU cores, improving the throughput of MRI image processing.

6. Distributed hospital network: Distributed computing services allow three hospitals to share patient records securely. Replication and local access improve availability during network failures.

Conflicting Requirements

Conflict	**Explanation**
Fast response vs. high throughput	Heavy workloads can delay time-critical alarms.
Reliability vs. maintenance	Software updates may interrupt database services.
Battery life vs. performance	Higher CPU usage consumes more energy.
Security vs. response time	Encryption and authentication add processing overhead.
Centralized access vs. availability	A central server or network failure may interrupt access to records.

Evaluation: The hospital should prioritize predictable response for patient monitors, energy efficiency for tablets, and reliability for the database server.

Requirements Critical to Patient Safety

- **Response time**: Alarms must meet the 10 ms deadline.
- **Reliability**: Patient-monitoring devices must operate consistently.
- **Data integrity**: Patient readings and medical records must remain accurate.
- **Availability:** Essential clinical records should remain accessible during failures.
- **Security:** Unauthorized users must not access or modify patient data.
- **Fault recovery**: Critical services must recover safely after failures.

Lessons from the History of Operating Systems

Historical OS type	**Application in the hospital**
Batch OS	Overnight billing and report generation.
Time-sharing OS	Multiple staff members accessing shared computing services.
Personal computer OS	Doctors' desktop workstations.
Mobile OS	Nurses' handheld tablets.
Real-time OS	Bedside patient monitors.
Multiprocessor OS	MRI imaging server.
Distributed computing	Record sharing between three hospitals.

Evaluation: The development of operating systems demonstrates that different computing environments require different designs. One general-purpose OS cannot automatically satisfy every hospital requirement.

Alternative Concept and Complexity Analysis

Alternative: Use a single general-purpose OS on every device instead of selecting a suitable OS for each environment.

A. Worst-case alarm response time

A general-purpose OS may experience unpredictable scheduling delays under heavy workloads. A properly configured and validated RTOS can provide a bounded response time.

The hospital's requirement is:

$$T_{\text{alarm}} \leq 10 \text{ ms}$$

The total time for processing, scheduling and raising the alarm must remain within this deadline.

B. Memory requirements

A general-purpose OS may require more memory and storage than a lightweight embedded RTOS. This can be inefficient for bedside monitors with limited resources. A mobile OS is more appropriate for tablets because it includes power-management features.

C. Administration effort

A single OS may simplify installation, staff training and update management. However, it may require additional configuration and testing to meet real-time, mobile and server requirements. Multiple OS types require separate maintenance procedures but are better suited to the different workloads.

D. Complexity analysis

Let n be the number of tasks.

- Linear task selection: O(n) time.
- Priority-queue insertion or removal: O(log n) time.
- Storage for n task records: O(n) space.

These complexities depend on the scheduling algorithm, not simply on the OS type. Actual alarm response time must be measured and verified rather than inferred from Big-O notation.

Conclusion

The hospital should use a combination of real-time, mobile, desktop, server and distributed computing environments. This design provides better patient safety, reliability, security, processing performance and energy efficiency than using one general-purpose OS everywhere. The design can also support future expansion to remote patient monitoring and cloud-based patient records.

TASK 2 – Judge the Type of Operating System for Each Environment

Comparison of Operating-System Types

OS type	Description	Hospital application
Mainframe OS	Handles large-scale data processing and many users.	Hospital administration and bulk record processing.
Desktop OS	Supports interactive applications and multitasking.	Doctors' workstations.
Multiprocessor OS	Uses multiple processors or CPU cores.	MRI imaging server.
Distributed system	Shares data and services across networked computers.	Connecting three hospitals.
Clustered system	Uses multiple servers for availability and failover.	Central database server.
Hard real-time OS	Requires critical tasks to meet strict deadlines.	Patient monitors with a 10 ms alarm deadline.
Soft real-time OS	Prioritizes timely responses, but occasional deadline misses are tolerable.	Non-critical medical video streaming.
Handheld / mobile OS	Supports portable devices and power management.	Nurses' tablets.

Best OS Type for Each Environment

Environment	Recommended OS	Justification	Rejected alternative
Bedside monitors	Hard real-time OS	Provides predictable scheduling for critical alarms.	General-purpose OS may not guarantee the deadline.
Nurses' tablets	Mobile OS	Provides battery management and wireless connectivity.	Desktop OS is less suitable for portable devices.
Doctors' workstations	Desktop OS	Supports clinical applications and multitasking.	RTOS is unnecessary for ordinary desktop tasks.
Database server	Clustered server OS	Supports failover and continuous access to records.	A single server creates a single point of failure.

MRI imaging server	Multiprocessor OS	Supports parallel processing of medical images.	A single-processor system may have lower throughput.
Partner hospital link	Distributed computing environment	Enables secure sharing of patient records.	An isolated system cannot directly share records.

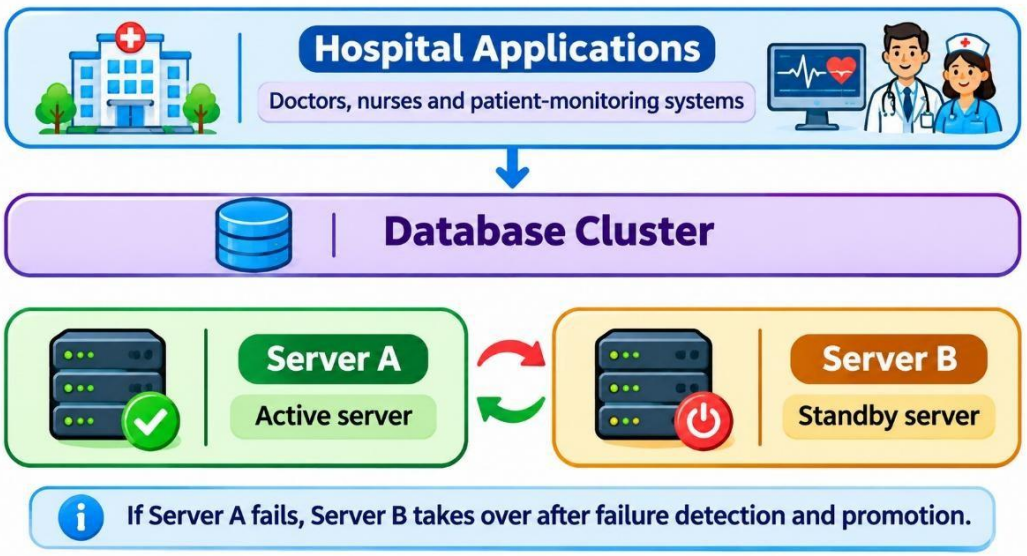
Symmetric vs. Asymmetric Multiprocessing

Symmetric Multiprocessing (SMP)	**Asymmetric Multiprocessing (AMP)**
Multiple processors share memory and execute tasks under one OS.	Processors have assigned roles or responsibilities.
Work can be distributed among available processors.	Tasks are allocated to specific processors or managed by a master processor.
Suitable for general-purpose MRI image processing.	Suitable for systems with specialized processing roles.

Recommendation: SMP is better for the MRI imaging server because multiple CPU cores can process image data in parallel, increasing throughput and improving resource utilization.

Clustered Database Server with Failover

A clustered system uses multiple servers to improve database availability.



Justification: A cluster reduces downtime and improves availability. Replication and automatic failover help maintain access to patient records. Failover within seconds is possible with suitable configuration and testing, but it is not guaranteed by clustering alone.

Alternative Concept and Complexity Analysis

Alternative: Use one large multiprocessor server instead of a cluster of smaller servers.

Factor	Single multiprocessor server	Clustered servers
Availability	A server failure may stop the service.	Another server can take over after a failure.
Processing time	Multiple processors improve parallel processing.	Work can be distributed across servers.
Memory	Limited to the machine's available memory.	Aggregate memory can grow by adding servers.
Scalability	Limited by the machine's maximum capacity.	Additional servers can be added.
Cost	May require expensive high-end hardware.	Requires multiple servers and cluster management.

Complexity analysis

Let n be the number of processors and m the number of requests.

- **Processing time**: Ideally, a parallel workload may approach (O(m/n), provided the work can be divided efficiently. Actual performance depends on communication overhead and contention.
- **Memory space:** A single server with memory capacity M has O(M) memory capacity. A cluster of k servers, each with capacity M, has aggregate physical memory capacity O(kM), although the memory is not necessarily shared.
- **Availability:** A single server is a single point of failure. A cluster improves availability through redundancy and failover.

Evaluation: A clustered database is preferable for the hospital because continuous access to patient records is more important than depending on one powerful machine.

Conclusion

The hospital should use a hard real-time OS for patient monitors, a mobile OS for tablets, a desktop OS for doctors' workstations, a multiprocessor OS for MRI processing, and a clustered server OS for the central database. Distributed computing services should connect the three hospitals. This design provides better patient safety, scalability, processing performance and availability than using one OS design for every environment.

TASK 3 – Evaluate Operating-System Structures

Comparison of Operating-System Structures

OS structure	Description	Advantages	Disadvantages
Monolithic	Most OS services run inside the kernel.	High performance and fast communication.	A faulty driver may crash the entire system.
Layered	The OS is divided into separate layers.	Easy to maintain, test and debug.	Communication between layers can add overhead.
Microkernel	Only essential services run in the kernel; other services run as separate processes.	Better fault isolation and reliability.	Message passing can increase overhead.
Modular	Supports loadable kernel modules, such as drivers and file systems.	Flexible and easier to extend.	A faulty kernel module can still crash the system.
Hybrid	Combines features of monolithic and microkernel designs.	Balances performance and modularity.	More complex to design and maintain.

Effect on Reliability, Performance and Updating

- **Monolithic:** Offers high performance, but a faulty kernel component can affect the entire OS.
- **Layered:** Improves maintainability through clear separation of functions, but layer communication may add overhead.
- **Microkernel**: Improves fault isolation because many services run outside the kernel, although message passing can reduce performance.
- **Modular:** Allows modules to be added or removed when needed, but kernel-level modules can still cause system failures.
- **Hybrid:** Combines efficient kernel operations with modular design, but its implementation is more complex.

Recommended Structures for the Hospital

Environment	Recommended structure	Justification
Bedside patient monitors	Microkernel-based RTOS, such as QNX Neutrino	Supports predictable real-time scheduling and service isolation.

Central database server	Modular monolithic Linux kernel	Provides high performance, hardware support and flexible kernel modules.

Justification: Patient monitors require predictable alarm responses and fault isolation. A microkernel-based RTOS is suitable if the complete device is validated to meet the 10 ms deadline. A modular Linux kernel is suitable for the database server because it supports efficient storage, networking and database workloads.

Structures Used by Linux, Windows, Android and Real-Time OSs

Operating system	**Structure**	**Explanation**
Linux	Modular monolithic	Core services run in kernel space, with loadable modules.
Windows	Hybrid	Combines kernel-mode components and separate system services.
Android	Modular monolithic	Uses the Linux kernel with Android-specific components and application services.
QNX Neutrino	Microkernel	Many OS services run as separate processes outside the kernel.
FreeRTOS	Small embedded kernel	Provides compact task scheduling and synchronization for embedded devices.

Alternative Concept and Complexity Analysis

Alternative: Use a monolithic kernel instead of a microkernel for bedside monitors.

A. Time cost of a service request

- Monolithic kernel: A system call can invoke services through direct in-kernel function calls, usually reducing communication overhead.
- Microkernel: A request may require message passing between separate processes, adding context switches and communication overhead.

B. Memory usage

A monolithic kernel may include more services and drivers in kernel space. A microkernel keeps the core kernel smaller, but additional memory is required for external service processes and message buffers.

Therefore, a smaller kernel does not necessarily mean lower total system memory usage.

C. Effect of a faulty driver

- Monolithic kernel: A faulty driver may crash the entire operating system.

- Microkernel: An isolated user-space driver may fail without crashing the kernel, provided suitable fault-recovery mechanisms are implemented.

D. Complexity Analysis

Let n be the number of service requests.

- **Monolithic structure:** Total processing time is O(n), assuming each request has bounded cost.
- **Microkernel structure:** Total processing time is also O(n) assuming bounded processing and message-passing costs.

Both have the same asymptotic complexity under these assumptions, but the microkernel may have greater constant overhead because of message passing and context switches.

Conclusion

A microkernel-based RTOS is suitable for bedside monitors because predictable response time and fault isolation are essential. A modular monolithic OS such as Linux is suitable for the central database server because of its performance and flexibility. These choices balance reliability, performance and maintainability according to the hospital's requirements.

TASK 4 – Evaluate the Services an Operating System Must Provide

Operating-System Services for Each Environment

OS service	**Patient monitors**	**Nurses' tablets**	**Doctors' workstations**	**Database server**
User interface	Minimal interface	Touch interface	Graphical interface	Admin interface
Program execution	Essential	Essential	Essential	Essential
I/O operations	Sensors and alarms	Touchscreen and wireless devices	Keyboard, mouse and medical devices	Storage devices
File systems	Minimal local storage	Essential	Essential	Essential
Communication	Sensor and alarm communication	Wi-Fi and hospital network	Hospital network	Database and network communication
Error detection	Critical	Essential	Essential	Critical

Resource allocation	Predictable scheduling	CPU and memory management	Multitasking	CPU, memory and storage management
Accounting	Minimal	Usage monitoring	User activity tracking	Resource monitoring and auditing
Protection and security	Essential	Essential	Essential	Critical

Services That Can Be Removed from Bedside Monitors

Bedside monitors have limited memory and processing capacity. Unnecessary services can be removed to improve efficiency.

- Full graphical desktop interface.
- Unnecessary background applications.
- Printing services.
- General-purpose multimedia services.
- Unused network protocols and device drivers.
- Unnecessary administrative utilities.

Essential services that must remain: Real-time scheduling, sensor input/output, alarm generation, essential communication, error detection, watchdog support and security controls.

Justification: Removing unnecessary services reduces memory usage and software complexity. However, patient-safety functions must never be removed simply to reduce the OS size.

Essential Services for the Database Server

The hospital database server requires the following services:

1. File-system management: Stores patient records and database files.
2. I/O operations: Reads and writes patient information.
3. Communication: Connects authorized hospital applications to the database.
4. Resource allocation: Manages CPU, memory and storage for concurrent users.
5. Protection and security: Prevents unauthorized access to patient records.
6. Error detection and recovery: Detects failures and supports recovery.
7. Program execution: Runs database and backup applications.
8. Accounting and auditing: Tracks resource usage and access to records.

Justification: These services are essential for secure, reliable and efficient management of patient records.

How the OS Protects One Patient's Records from Another User's Programs

The OS and database application use several security mechanisms:

- Authentication: Verifies the identity of each user.
- Access permissions: Restrict access to files and resources.
- Process isolation: Prevents processes from accessing protected memory belonging to other processes.
- Role-based access control (RBAC): Gives doctors, nurses and administrators appropriate permissions.
- Encryption: Protects records during storage and transmission.
- Audit logs: Record access and changes for security monitoring.

Example: A nurse can access assigned patients' records but cannot modify database configuration or view unrelated records without authorization.

The OS provides basic protection, while the database and hospital application must enforce patient-level access permissions.

Alternative Concept and Complexity Analysis

Alternative: Use a full desktop OS on nurses' tablets instead of a reduced handheld OS.

Factor	**Full desktop OS**	**Reduced handheld OS**
Memory usage	Generally higher because of additional services	Lower when unnecessary services are excluded
Storage usage	Larger installation footprint	Smaller installation footprint
Start-up time	May take longer to start additional services	Can start faster with fewer services
Battery life	Background services may increase power consumption	Optimized for mobile power management
Security	Strong security is possible but needs configuration	Includes mobile-oriented security controls
Maintenance	More services may need updates	Smaller software footprint may simplify maintenance

Complexity analysis

Let S represent the number of enabled services.

- **Memory space:** Depends on the kernel, applications and memory required by each enabled service.
- **Storage space:** Generally increases as additional services and applications are installed.

- **Start-up time:** May increase with the number of services that must initialize, although some services start in parallel.
- Battery consumption: More background activity can increase CPU usage and energy consumption.

Exact memory usage, start-up time and battery life depend on the hardware and configuration. Both operating systems should be tested on the same tablet to make a reliable comparison.

Conclusion

The hospital should provide only the services required by each environment. Bedside monitors need a small, predictable and reliable set of real-time services, while database servers require comprehensive storage, communication, security and recovery services. Nurses' tablets should use a mobile OS with efficient power management. This approach improves performance, protects patient records and reduces unnecessary resource consumption.

TASK 5 – Examine System Calls

Introduction

A system call is a mechanism through which a user program requests services from the operating system kernel. System calls allow programs to access files, create processes, communicate with other programs and manage hardware safely.

Categories of System Calls

Category	Purpose	Examples
Process control	Creates, executes and terminates processes.	fork(), exec(), exit(), wait()
File management	Opens, reads, writes and closes files.	open(), read(), write(), close()
Device management	Accesses and controls devices.	ioctl(), read(), write()
Information maintenance	Retrieves system information.	getpid(), time(), uname()
Communication	Exchanges data between processes.	pipe(), socket(), send(), recv()
Protection	Controls permissions and access.	chmod(), chown(), setuid()

Steps When a Program Calls read()

When a program reads data from a file, the following steps occur.

1. User program calls read()

2. C library wrapper prepares the system call

3. Trap / system-call instruction switches to kernel mode

4. Kernel validates arguments and reads data

5. Kernel returns the byte count or an error

6. Program continues in user mode

Explanation:

1. The program invokes read(fd, buffer, count).
2. The C library wrapper prepares the system-call number and arguments.
3. A system-call instruction transfers control to the kernel.
4. The kernel checks permissions and performs the requested I/O.
5. The kernel returns the number of bytes read, 0 at end-of-file, or -1 on error.
6. The program resumes execution in user mode.

How parameters reach the kernel

System-call parameters may be passed through CPU registers, a memory block referenced by a register, or the stack, depending on the architecture and calling convention. For read(), the arguments are the file descriptor, buffer address and number of bytes.

Using strace to Observe System Calls

The Linux strace utility displays the system calls made by a program.

Example C program:

```
#include <fcntl.h>
#include <unistd.h>
int main(void) {
   char buffer[100];
   int fd = open("sample.txt", O_RDONLY);
   if (fd < 0)
      return 1;
   ssize_t n = read(fd, buffer, sizeof(buffer));
   if (n > 0)
      write(STDOUT_FILENO, buffer, (size_t)n);
   close(fd);
   return 0;
```

}

Expected evidence: The output will show calls such as openat(), read(), write() and close(). Linux commonly uses openat() internally for file-opening requests, even when the program calls the C library function open().

Alternative Concept and Complexity Analysis

Alternative: Use C library functions fopen(), fread() and fwrite() instead of directly using open(), read() and write().

Feature	**Direct system calls**	**C library functions**
Functions	open(), read(), write()	fopen(), fread(), fwrite()
Buffering	Managed by the program or kernel	Library buffering is available
System-call frequency	May be high with small transfers	Can be reduced by buffering
Control	More direct control over file descriptors	Convenient stream-based interface

System-call count

For a file containing n bytes and a transfer buffer of b bytes, the approximate number of read() calls is:

$$N_{\text{read}} = \lceil \frac{n}{b} \rceil + 1$$

The additional call detects the end-of-file when the file size is an exact multiple of the buffer size. This formula assumes successful reads of the requested amount, which is typical for ordinary file reads but not guaranteed for every file type.

For a 4096-byte file and a 1024-byte buffer, approximately four successful reads plus one end-of-file read are needed.

Time and space complexity

- Time complexity: O(n)for reading n bytes, excluding device latency and other overhead.
- Buffer space: O(b) for a buffer of size b
- Library buffering: Can reduce the number of system calls by collecting data into larger transfers, but uses additional memory.

Conclusion

System calls provide the interface between user programs and the OS kernel. The read() example demonstrates how control transfers to the kernel and returns to the program. Using strace helps verify the actual system calls. Library buffering can improve file-I/O efficiency by reducing system-call frequency, although it requires additional buffer memory.

TASK 6 – Evaluate the Cost of System Calls

Why Does a System Call Cost More Than an Ordinary Function Call?

An ordinary function call usually transfers control to another function within the same program. A system call requests a service from the operating system kernel.

A system call generally involves:

1. Switching execution from user mode to kernel mode.
2. Validating the system-call number and arguments.
3. Performing the requested operation, such as file I/O.
4. Returning the result and restoring user-mode execution.

Therefore, system calls have additional overhead compared with ordinary function calls. For a hospital database backup, reducing unnecessary system calls can improve copying efficiency.

Effect of Buffer Size on System Calls

A buffer temporarily stores data while it is copied from a source file to a destination file.

For a 10,000-byte file, the expected number of read() calls is:

$$N_{\text{read}} = \lceil \frac{10000}{b} \rceil + 1$$

Here, b is the buffer size, and the additional call detects the end of the file.

Buffer size	Successful reads	EOF read	Total read() calls
1 byte	10,000	1	10,001
512 bytes	20	1	21
4096 bytes	3	1	4

Evaluation: A larger buffer reduces the number of system calls, which generally lowers overhead. However, excessively large buffers consume more memory, so the best buffer size should be selected through measurements.

program:

```
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>

#include <fcntl.h>

#include <unistd.h>

long copy_file(const char *src, const char *dst, size_t bufsize)

{

   char buf[8192];
```

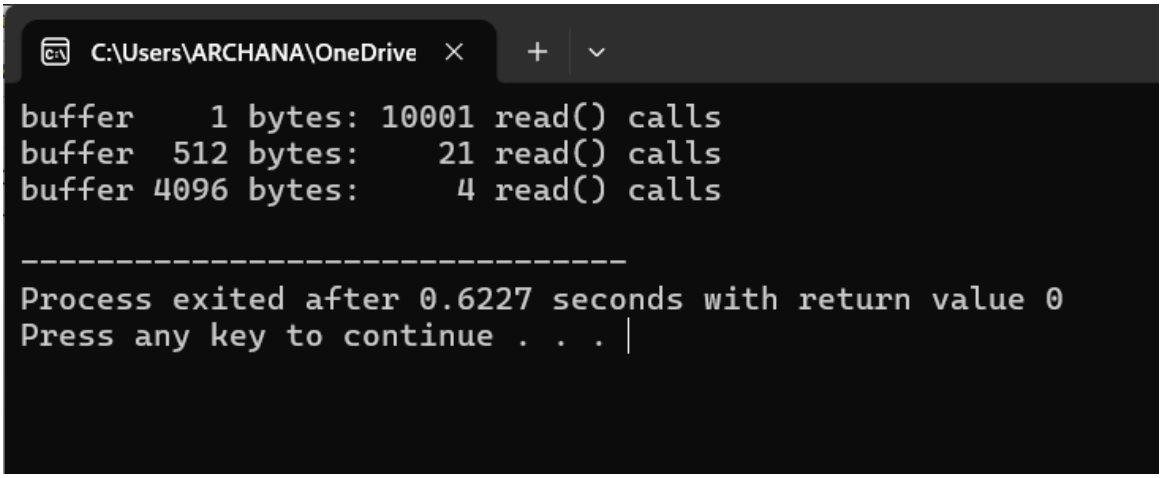
```
int in = open(src, O_RDONLY);
if (in < 0) return -1;
int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (out < 0) { close(in); return -1; }
long calls = 0;
ssize_t n;
while (1) {
    n = read(in, buf, bufsize);
    calls++;
    if (n == 0)
        break;
    if (n < 0) {
        close(in);
        close(out);
        return -1;
    }
    size_t written = 0;

    while (written < (size_t)n) {
        ssize_t w = write(out, buf + written,
                          (size_t)n - written);
        if (w <= 0) {
            close(in);
            close(out);
            return -1;
        }
        written += (size_t)w;
    }
}
```

```
    close(in);
    close(out);
    return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w");
    if (f == NULL) return 1;
    for (int i = 0; i < 10000; i++)
        fputc('A' + i % 26, f);
    fclose(f);
    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++) {
        long calls = copy_file("records.dat", "copy.dat",
                               sizes[i]);
        printf("buffer %4zu bytes: %5ld read() calls\n",
               sizes[i], calls);
    }
    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

Expected output:



These results assume successful reads of the expected sizes and one additional call to detect EOF.

Verify using strace

Install strace if necessary, then execute:

strace -c -e trace=read ./task6

The program prints its own read() counts. The strace summary reports the system-wide count for the traced process, which may include reads outside copy_file(). To inspect individual calls, use:

strace -e trace=read ./task6

The trace provides evidence that the number of read() calls decreases as the buffer size increases.

Alternative Concept and Complexity Analysis

Alternative: Use memory-mapped files (mmap()) instead of repeatedly calling read() and write().

Factor	**read() / write()**	**Memory-mapped files**
System calls	Repeated calls based on buffer size	Mapping and unmapping calls, with possible additional calls
Data transfer	Explicitly copies data through a user-space buffer	Accesses file data through a mapped memory region
Memory usage	Usually requires a buffer of size b	Uses virtual address space proportional to the mapped region
Performance	Small buffers can cause high overhead	Can reduce explicit copying and system-call frequency
Limitations	Simple and predictable buffer management	Page faults, mapping overhead and storage synchronization still matter

Big-O analysis

Let n be the file size and b the buffer size.

- Time complexity: Both approaches generally require O(n) work to process the file.
- Buffered copying space: O(b)additional user-space buffer memory.
- Memory mapping space: O(n)virtual address space for mapping the complete file, although not all pages must be resident in physical memory at once.

Memory mapping can reduce explicit system-call overhead, but it does not eliminate page faults or storage I/O. For very large files, mapping the entire file may also be impractical on systems with limited address space.

Best Buffer Size for the Hospital Backup Program

Among the three tested sizes, 4096 bytes is the best initial choice because it requires only four read() calls for the 10,000-byte test file, compared with 21 calls for 512 bytes and 10,001 calls for one byte.

However, the final choice should be based on measured execution time, memory consumption and throughput using representative hospital backup files. A larger buffer may perform better, but this must be verified experimentally.

Conclusion

System calls have additional overhead because they transfer control to the OS kernel. Increasing buffer size reduces the number of read() calls and can improve file-copy performance. For the given test file, a 4096-byte buffer provides the lowest call count among the tested sizes. Memory-mapped files are an alternative for reducing explicit system calls, but their performance and memory trade-offs must also be evaluated.

TASK 7 – Evaluate Process Control System Calls

Functions of Process Control System Calls

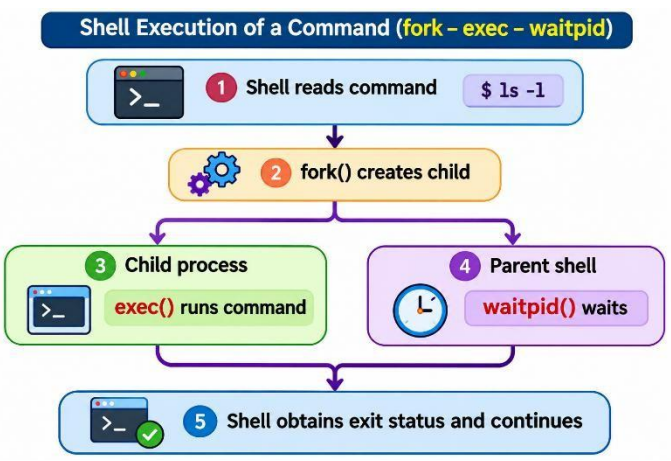
System call	Purpose
fork()	Creates a child process by duplicating the calling process.
exec()	Replaces the current process image with a new program.
wait() / waitpid()	Waits for a child process to change state and collects its exit status.
exit()	Terminates a process and provides an exit status to the parent.

How a Shell Runs a Command

A command shell generally follows these steps:

1. The shell reads a command entered by the user.
2. It calls fork() to create a child process.
3. The child calls exec() to run the requested program.
4. The parent shell calls waitpid() to wait for the child to finish.
5. The shell retrieves the child's exit status and continues to the next command.

Process flowchart:



Zombie and Orphan Processes

- **Zombie process**: A child has terminated, but its parent has not yet collected its exit status using wait() or waitpid(). The OS retains a process-table entry until the status is collected.
- **Orphan process:** A child continues running after its parent terminates. The OS reparents it to an appropriate system process, which can later collect its exit status.

Evaluation: A shell should use waitpid() to collect child exit statuses and prevent terminated children from remaining zombies.

Why Use fork() Followed by exec()?

fork() creates a separate child process, and exec() replaces the child's program with the requested command.

This approach lets the parent shell continue to exist while the child executes a different program. It also allows the child to configure input/output redirection before calling exec().

A single fork()-like operation would not itself load and execute an entirely different program in the same way.

Program:

```
#include <stdio.h>

#include <stdlib.h>

#include <windows.h>

int main(void)

{

   STARTUPINFOA si;

   PROCESS_INFORMATION pi;

   DWORD exitStatus;

   ZeroMemory(&si, sizeof(si));

   si.cb = sizeof(si);

   ZeroMemory(&pi, sizeof(pi));

   printf("shell: running job 1\n");

   fflush(stdout);

   char command1[] = "cmd.exe /c set /a 6*7";

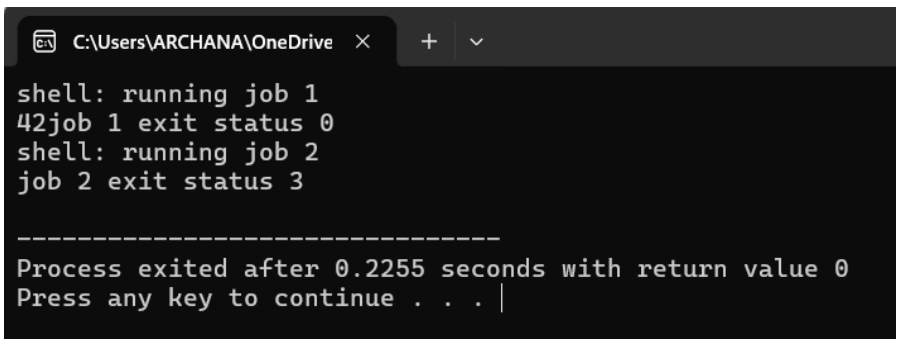
   if (!CreateProcessA(

        NULL, command1, NULL, NULL, FALSE,

        0, NULL, NULL, &si, &pi)) {
```

```
        printf("Failed to start job 1\n");
        return 1;
    }
    WaitForSingleObject(pi.hProcess, INFINITE);
    GetExitCodeProcess(pi.hProcess, &exitStatus);
    printf("job 1 exit status %lu\n",
           (unsigned long)exitStatus);
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
    printf("shell: running job 2\n");
    fflush(stdout);
    char command2[] = "cmd.exe /c exit 3";
    if (!CreateProcessA(
            NULL, command2, NULL, NULL, FALSE,
            0, NULL, NULL, &si, &pi)) {
        printf("Failed to start job 2\n");
        return 1;
    }
    WaitForSingleObject(pi.hProcess, INFINITE);
    GetExitCodeProcess(pi.hProcess, &exitStatus);
    printf("job 2 exit status %lu\n",
           (unsigned long)exitStatus);
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
    return 0;
}
```

Expected output



Explanation: The expr command prints 42 and exits successfully with status 0. The second child explicitly terminates with status 3.

Why Is stdout Flushed Before fork()?

stdout may be buffered. If pending output is not flushed before fork(), the child inherits a copy of the parent's output buffer. Both processes may later flush the same buffered text, causing duplicate output.

Calling fflush(stdout) before fork() ensures pending output is written once before the process is duplicated.

What happens if the second flush is removed and exit(3) replaces _exit(3)?

When output is redirected to a file, stdout is normally fully buffered. If the second fflush(stdout) is removed, the text shell: running job 2 may remain in the buffer when fork() occurs.

The child inherits that buffer. If it calls exit(3), the C library flushes the inherited buffer before terminating. The parent also retains its copy and later flushes it, so the line may appear twice in the file.

_exit(3) bypasses normal C library flushing and avoids this particular duplicate-buffer effect.

Alternative Concept and Complexity Analysis

Alternative: Use posix_spawn() or threads instead of fork() followed by exec().

Factor	**fork() + exec()**	**posix_spawn()**	**Threads**
Process creation	Creates a child, then replaces its program	Creates a process and starts the specified program	Creates a thread within the same process
Memory	Uses copy-on-write on typical Linux systems	Can avoid some of the overhead of a separate fork step	Shares the process address space
Isolation	Separate process address space	Separate process address space	Shared memory and resources
Suitability for shell commands	Suitable	Often efficient for launching commands	Less suitable for independent external commands

Complexity analysis

Let k be the number of commands launched.

- Time: Launching and waiting for each command incurs process-creation and execution overhead. The total time depends on the commands and system load; it cannot be described accurately by O(k) alone without assuming bounded command execution time.
- Memory: With fork(), copy-on-write usually avoids immediately copying every memory page, but page tables and other process resources still require memory.

- Threads: Thread creation may be cheaper, and threads share memory, but they do not provide the same process isolation as independent commands.

Conclusion

The fork(), exec() and waitpid() system calls enable a shell to launch commands, execute programs and collect exit statuses. Flushing output before fork() prevents duplicated buffered output. For launching external hospital administration commands, posix_spawn() can be an alternative, while independent processes offer better isolation than threads.

TASK 8 – Evaluate CPU States and Protection

User Mode and Kernel Mode

The CPU operates in two modes to protect the hospital's operating system.

- User mode: Applications such as doctors' software and patient-monitoring programs run with restricted privileges.
- Kernel mode: The operating system performs privileged operations, such as managing memory, devices and interrupts.
- Mode bit: A hardware indicator distinguishes user mode from kernel mode. Privileged instructions can execute only in kernel mode.

Evaluation: This separation prevents ordinary applications from directly controlling critical hardware or modifying the operating system.

Interrupts, Traps and Timer Interrupts

- Interrupt: A hardware event, such as a sensor signal, requests CPU attention.
- Trap: A software-generated event, such as a system call or an exception.
- Timer interrupt: Periodically returns control to the operating system so it can schedule processes and prevent one process from monopolizing the CPU.

If a program could disable interrupts indefinitely, a patient monitor might fail to process sensor events promptly, delaying alarms and risking patient safety. Therefore, interrupt control must be restricted to trusted kernel-level code.

Memory Protection

The Memory Management Unit (MMU) checks memory accesses using page tables and access permissions. A program cannot read or modify another program's protected memory without authorization. Invalid accesses cause an exception handled by the operating system.

For example, a nurse's application must not access another patient's protected records directly.

Alternative Concept: Polling Instead of Interrupts

Polling repeatedly checks a sensor in a loop, whereas interrupts notify the CPU when an event occurs.

Factor	Interrupts	Polling
Response time	Responds after interrupt detection and handling	May wait until the next poll
Worst-case detection delay	Depends on interrupt latency and system load	Approximately the polling interval, plus processing delay
CPU usage	Usually lower when events are infrequent	Can waste CPU time checking repeatedly
Power consumption	Generally lower during idle periods	May consume more battery power
Suitability	Preferred for event-driven patient monitoring	Useful for predictable, frequent checks in some systems

If the polling interval is 5 milliseconds, an abnormal reading occurring just after a check may wait nearly 5 milliseconds before detection, excluding processing delays. Interrupts can provide faster notification, but their latency must also be bounded to meet the hospital's 10-millisecond alarm deadline.

Conclusion

CPU modes, interrupts and memory protection are essential for hospital safety. A real-time operating system with controlled interrupt handling and hardware memory protection is preferable for bedside monitors. Polling may be suitable for selected tasks, but its delay and power consumption must be evaluated before use in critical monitoring.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems

Levels of Memory Hierarchy

Memory hierarchy organizes storage according to speed, capacity and cost.

Memory level	Typical access time	Capacity	Purpose
CPU registers	Less than 1 ns (approx.)	Bytes to KB	Store immediate CPU data
Cache memory	About 1–10 ns	KB to tens of MB	Store frequently used data
Main memory (RAM)	About 50–100 ns	GBs	Hold active programs and records
SSD	About 50–200 μs for typical access	GBs to TBs	Store databases and MRI images

Hard disk (HDD)	About 5–15 ms for random access	TBs	Long-term storage and backups
Network storage	About 0.1–10 ms or more	Scalable	Share records across hospitals

Where Hospital Data Should Be Stored

Data type	Recommended storage	Justification
MRI images	High-capacity SSD or dedicated imaging storage	Large files require high throughput for fast image processing.
Patient records	SSD-backed database with redundant storage	Fast retrieval, reliability and data protection are essential.
Real-time monitor readings	RAM buffers and local persistent storage	Fast access supports timely alarms; persistent storage preserves readings.
Hospital backups	Separate backup storage, including off-site or cloud copies	Helps recover data after hardware failure or disasters.
Shared hospital records	Secure network or cloud storage with access controls	Allows authorized access across partner hospitals.

Programmed I/O, Interrupt-Driven I/O and DMA

Method	Working	Evaluation
Programmed I/O	CPU repeatedly checks device status and transfers data.	Simple, but wastes CPU time.
Interrupt-driven I/O	Device interrupts the CPU when service is required.	Reduces continuous polling but may generate many interrupts.
Direct Memory Access (DMA)	A DMA controller transfers blocks directly between a device and memory.	Efficient for large data transfers with less CPU involvement.

Best choice for MRI scanners: DMA. MRI images are large, so DMA transfers blocks of data to memory without requiring the CPU to handle every word individually. Interrupts can notify the CPU when a transfer completes.

Effect of Caching on Patient Record Lookup

Caching stores frequently accessed data in faster memory.

- A cache hit occurs when the required record is available in the cache, reducing access time.
- A cache miss occurs when the record must be fetched from slower memory or storage.
- Database caching can improve response time for frequently accessed patient records.

- Sensitive records must be protected through access controls, and cached data must be invalidated or updated when the original data changes.

Therefore, caching improves performance, but it must not return outdated or unauthorized patient information.

Alternative Concept and Complexity Analysis: Interrupt-Driven I/O vs DMA

Factor	**Interrupt per word**	**DMA**
Number of interrupts	Approximately 131 million	Usually a small number of completion or block interrupts
CPU overhead	Extremely high	Much lower
CPU availability	CPU spends substantial time servicing interrupts	CPU can perform other processing during transfer
Buffering	Requires data buffers and frequent CPU handling	Uses memory buffers and DMA descriptors
Suitability	Poor for large MRI transfers	Highly suitable for large transfers

If each interrupt requires 1 microsecond of CPU time, servicing 131,072,000 interrupts would require approximately 131 seconds of CPU time, ignoring other overhead. DMA greatly reduces this interrupt-handling burden.

Complexity analysis

- Interrupt per word: O(n)interrupt-handling operations for n words.
- DMA: With a fixed-size block of B bytes, approximately O(n/B) block operations or completions may be needed, depending on the hardware and driver design.
- Memory usage: Both methods require buffers; DMA typically uses block buffers rather than requiring a separate buffer for every word.

Conclusion

A smart hospital should use a suitable memory hierarchy, SSD-backed databases, reliable backup storage and caching for frequently accessed records. Interrupt-driven I/O is useful for event notifications, while DMA is the preferred method for transferring large MRI images because it reduces CPU overhead and improves overall system efficiency.

TASK 10 – Measure and Compare

Introduction

The performance of operating-system programs can be evaluated by measuring system calls, CPU time, process creation time and thread creation time on Linux. These measurements help determine whether the chosen OS design is efficient for a smart hospital.

Measure System Calls for Different Buffer Sizes

Use the copy program developed in Task 6. Run the program under strace -c to count system calls.

gcc -std=c11 -Wall -Wextra task6.c -o task6

strace -c -e trace=read,write ./task6

For more detailed results, test each buffer size separately so that the counts can be compared accurately.

Expected read() counts for the 10,000-byte test file:

Buffer size	Expected read() calls	Evaluation
1 byte	10,001	Very high system-call overhead
512 bytes	21	Lower overhead
4096 bytes	4	Lowest count among these options

A larger buffer reduces the number of system calls, but the best buffer size should be selected using actual execution-time measurements.

Measure User-Mode and Kernel-Mode CPU Time

Run the following command:

/usr/bin/time -p ./task6

Example output format:

real 0.02

user 0.00

sys 0.01

- Real time: Total elapsed time.
- User time: CPU time spent executing user-space code.
- System time: CPU time spent executing kernel operations.

The values above are illustrative; record the actual values from your Linux system.

Compare the Creation of 1,000 Processes and 1,000 Threads

```
#include <stdio.h>

#include <windows.h>

#define COUNT 1000

static DWORD WINAPI process_function(LPVOID arg)

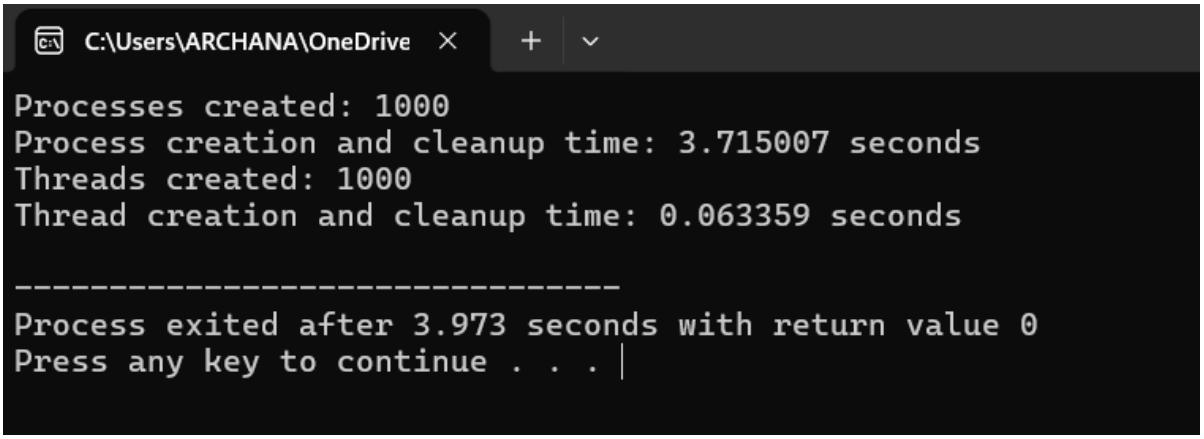
{
```

```
    (void)arg;
    return 0;
}
static DWORD WINAPI thread_function(LPVOID arg)
{
    (void)arg;
    return 0;
}
static double elapsed(LARGE_INTEGER start,
                      LARGE_INTEGER end,
                      LARGE_INTEGER frequency)
{
    return (double)(end.QuadPart - start.QuadPart)
          / frequency.QuadPart;
}
int main(void)
{
    LARGE_INTEGER start, end, frequency;
    HANDLE handles[COUNT];
    int created;
    int i;
    QueryPerformanceFrequency(&frequency);
    /* Measure process creation and cleanup */
    created = 0;
    QueryPerformanceCounter(&start)
    for (i = 0; i < COUNT; i++) {
        STARTUPINFOA si;
        PROCESS_INFORMATION pi;
        char command[] = "cmd.exe /c exit 0";
```

```
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));
    if (!CreateProcessA(
            NULL, command, NULL, NULL, FALSE,
            CREATE_NO_WINDOW, NULL, NULL, &si, &pi)) {
        printf("Process creation failed at %d\n", i);
        break;
    }
    CloseHandle(pi.hThread);
    handles[created++] = pi.hProcess;
}
for (i = 0; i < created; i++) {
    WaitForSingleObject(handles[i], INFINITE);
    CloseHandle(handles[i]);
}
QueryPerformanceCounter(&end)
printf("Processes created: %d\n", created);
printf("Process creation and cleanup time: %.6f seconds\n",
       elapsed(start, end, frequency));
/* Measure thread creation and cleanup */
created = 0;
QueryPerformanceCounter(&start)
for (i = 0; i < COUNT; i++) {
    handles[created] = CreateThread(
        NULL, 0, thread_function, NULL, 0, NULL);
    if (handles[created] == NULL) {
        printf("Thread creation failed at %d\n", i);
        break;
    }
```

```
        created++;
    }
    for (i = 0; i < created; i++) {
        WaitForSingleObject(handles[i], INFINITE);
        CloseHandle(handles[i]);
    }
    QueryPerformanceCounter(&end);
    printf("Threads created: %d\n", created);
    printf("Thread creation and cleanup time: %.6f seconds\n",
           elapsed(start, end, frequency));
    return 0;
}
```

Output:



Alternative Concept and Complexity Analysis

If threads were used instead of processes, the results would generally be as follows:

Factor	**Processes using fork()**	**Threads using pthread_create()**
Creation time	Usually higher	Usually lower
Memory	Separate address spaces; copy-on-write can reduce physical copying	Shared address space, with separate thread stacks
Communication	Requires IPC mechanisms for explicit data exchange	Can share data directly
Isolation	Stronger isolation between processes	A faulty thread can affect the whole process

Suitability	Better for independent, isolated tasks	Better for concurrent tasks sharing data

Big-O analysis

- Creating n processes requires O(n) creation operations.
- Creating n threads requires O(n) creation operations.
- The actual execution time depends on the OS, hardware, resource limits and scheduling overhead.
- Process memory is not necessarily fully duplicated because Linux commonly uses copy-on-write.
- Threads share the process address space, reducing duplicated memory, but each thread still needs its own stack and management resources.

Measurement Results Table

Fill this table with the actual results from your Linux execution.

Experiment	**Measurement**	**Your result**
1-byte buffer	read() calls	10,001 expected
512-byte buffer	read() calls	21 expected
4096-byte buffer	read() calls	4 expected
Copy program	Real, user and system time	Record actual values
1,000 processes	Creation and cleanup time	Record actual values
1,000 threads	Creation and cleanup time	Record actual values

Conclusion

Measurements help evaluate OS performance rather than relying only on theory. Larger buffers reduce system-call frequency, while threads generally require less creation overhead and share memory more efficiently than separate processes. However, the final conclusions should be based on actual Linux measurements, because performance varies with hardware, system load and resource limits.

TASK 11 – Recommend and Defend the Operating-System Plan

Recommended Operating-System Plan

For the 500-bed smart hospital, I recommend a hybrid operating-system architecture that uses different OS types according to each environment's requirements. A single general-purpose OS is not suitable for every device because real-time alarms, battery life, database availability and high-performance image processing have different needs.

OS Type, Structure, Services and I/O Methods

Hospital environment	Recommended OS type and structure	Services and I/O method
Bedside patient monitors	Real-time OS (FreeRTOS or QNX); microkernel preferred where suitable	Priority scheduling, interrupt handling, watchdogs and sensor interrupts
Nurses' handheld tablets	Android; hybrid OS structure	Touch interface, power management, Wi-Fi, secure authentication
Doctors' workstations	Windows or Linux desktop OS; hybrid or modular structure	GUI, multitasking, file management and network communication
Patient-record database	Linux server OS; modular kernel with a high-availability cluster	Access control, logging, redundant storage and failover
MRI imaging server	Linux multiprocessor OS with symmetric multiprocessing (SMP)	Multicore processing, high-throughput storage and DMA
Partner hospitals and cloud	Distributed system using secure network services	Encrypted communication, replication and access control

Protection, Reliability and Limitations

- **Patient safety**: Real-time scheduling should ensure that abnormal readings trigger alarms within the required 10 milliseconds under validated worst-case conditions.
- **Database availability**: A clustered database with replicated data and automatic failover reduces downtime. Backups are also required.
- **Security:** Authentication, role-based access, encryption, audit logs and memory protection prevent unauthorized access to patient records.
- **Performance:** DMA supports efficient MRI image transfers, while caching improves frequently accessed database queries.
- **Limitations:** Different operating systems increase maintenance and staff-training costs. Failover can still fail, and network or cloud outages may affect remote access.

Suitability for Three Hospitals

The plan remains suitable when the system expands to three hospitals because distributed services can share authorized patient records. Centralized identity management, secure APIs, data replication and monitoring should be implemented.

Each hospital should retain essential local monitoring and emergency functions so that a network failure does not prevent bedside alarms. Cloud services should supplement—not replace—critical local safety functions.

Alternative Concept: Virtual Machines on Two Large Servers

If all services were moved into virtual machines (VMs) on two large physical servers, the arrangement could reduce hardware costs and simplify centralized administration.

Factor	Separate physical machines	Two physical servers running VMs
Alarm response	Dedicated resources can provide predictable timing	CPU contention and virtualization overhead may affect timing
Memory	Dedicated memory per machine	Memory is shared among VMs and must be allocated carefully
Processor use	Hardware dedicated to each workload	Better consolidation, but competing workloads can reduce performance
Reliability	Failure affects the relevant machine	A host failure may affect many VMs unless failover is configured

Evaluation: Virtualization is appropriate for database replicas, application servers, development systems and analytics. However, bedside patient monitors should remain on dedicated embedded hardware with a validated real-time OS. Critical services should have resource reservations, redundancy and tested recovery procedures.

Evidence and Measurement-Based Justification

The final deployment should be validated using the measurements collected in Tasks 6 and 10.

Measurement	Evidence to use	Decision supported
System-call count	Compare 1-byte, 512-byte and 4096-byte buffers	Select an efficient backup buffer
CPU time	Compare user and kernel time	Identify system-call overhead
Process and thread creation time	Compare 1,000 processes with 1,000 threads	Choose suitable concurrency mechanisms
Alarm latency	Measure worst-case sensor-to-alarm response under load	Verify the 10 ms requirement
Failover time	Simulate a database-server failure	Verify recovery and availability

Conclusion

A hybrid OS architecture is the best choice for the smart hospital. A real-time OS protects bedside alarm responsiveness, Android supports handheld mobility, desktop systems support clinical work, Linux clusters provide database availability, and multiprocessor Linux servers handle MRI processing. Virtualization can improve efficiency for suitable server workloads, while critical monitoring functions should remain locally available and independently validated.

TASK 12 – Re-evaluate and Adapt the Design

A. Impact Analysis

The hospital has introduced two new requirements:

1. Remote patient-monitoring devices in patients' homes that send readings through the internet.
2. A cloud-based patient-record service shared with partner hospitals.

These changes affect the original operating-system design as follows.

Component	Impact
Home monitoring devices	Require a lightweight real-time OS for fast sensor processing and local alarms.
Communication	Requires secure internet connectivity and reliable data transmission.
Cloud database	Requires a distributed system for sharing patient records across hospitals.
Security	Requires encryption, authentication and access control.
Reliability	Local monitoring must continue even if the internet connection fails.
Scalability	Cloud resources must support increasing numbers of patients and devices.

Evaluation: The original design supports hospital operations, but additional cloud, networking and security services are needed for remote monitoring.

B. Evaluation of the Original Design

The original design uses a real-time OS for bedside monitors, Android for tablets, desktop OSs for workstations, Linux clustering for the database and multiprocessor Linux for MRI processing.

Requirement	Evaluation
Response time	Local real-time processing supports quick alarms without waiting for internet communication.
Reliability	Local monitoring continues during network failures, but remote transmission may be delayed.
Security	Existing protection must be extended to home devices and cloud services.
Scalability	Cloud infrastructure can support more users and additional hospitals.
Availability	Database replication and backups improve recovery, but cannot guarantee zero downtime.

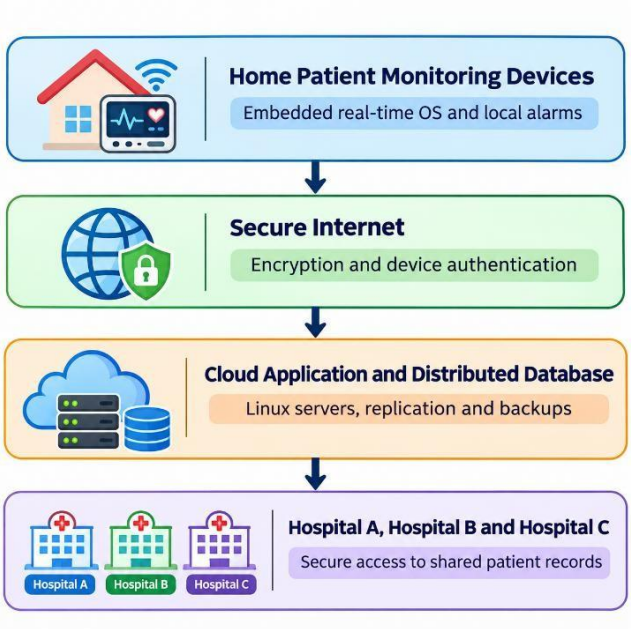
Judgment: The original design is suitable as a foundation, but it cannot fully support the new requirements without modification.

C. Adaptation or Redesign

Decision: MODIFY the original design.

The existing operating systems will be retained, and secure home monitoring and cloud services will be added.

Revised system architecture



The modifications are:

1. **Home devices**: Use an embedded real-time OS to process readings and trigger local alarms.
2. **Secure communication:** Encrypt data transmitted over the internet and authenticate each device.
3. **Cloud storage**: Use a distributed database with replication, backups and recovery mechanisms.
4. **Access control:** Allow only authorized doctors and nurses to access patient records.
5. **Scalability:** Increase cloud storage and processing resources as the number of users grows.

D. Evidence-Based Defence

The decision to modify the design is supported by the following analysis.

Evidence	Justification
Alarm deadline	Local processing avoids internet delays. The 10 ms deadline must be verified through worst-case testing.
Task 6 measurements	For a 10,000-byte file, a 4096-byte buffer requires 4 expected read() calls, compared with 10,001 calls for a 1-byte buffer. This supports efficient data transfer.
Task 10 measurements	Process and thread creation times help evaluate server resource requirements when actual measurements are collected.
Network failure	Local alarms continue, and readings can be queued until connectivity returns.
Security	Encryption, authentication and access controls protect patient information.
Cost	Cloud services improve scalability but introduce recurring costs and network dependence.

Trade-off: Cloud services make record sharing easier, but internet failures can delay remote data transmission. Therefore, critical alarms must operate locally rather than depend on cloud availability.

Conclusion

The hospital should modify its original design instead of retaining it unchanged or completely redesigning it. Real-time operating systems will continue to support critical monitoring, while secure internet communication and distributed cloud databases will enable home monitoring and record sharing across three hospitals. This approach improves scalability and availability while preserving patient safety and data security.